TrustVault Threat Model

THREATS | Snovasys Fresher Hiring Challenge 2026

1. Purpose

When I started this project I thought security meant adding a login page. While planning, I understood it is much more than that. Before writing code I had to ask a few questions. What am I protecting? Who may want it? How could they get it? What stops them? What happens if that protection fails? This document is my answer to those questions.

I tried to keep this document truthful. For each threat I wrote what I did, how it can be checked and what is still weak. I did not want to write that everything is solved, because that would not be true. The controls listed here are the ones in the backend, and they do not prove that the application has no vulnerability at all.

2. How I made this

1. I described the system and marked the places where data crosses from one part to another, like browser to API and API to database.
2. I listed the things that need protection.
3. I listed the people who may attack and what they could try.
4. For each realistic threat I wrote the scenario, the damage, the control I have, and the risk that remains.
5. I gave a rough rating from my own judgement and decided what to fix first.

I also compared my list with the OWASP API Security Top 10. It helped me notice object level authorization, which means one user reaching another user's object by changing an ID.

3. System overview

TrustVault has a React and Vite frontend, a Node.js and Express backend, a MySQL database and a private folder on the server for the uploaded files. Login uses JWT tokens and passwords are hashed with bcryptjs. Sharing is controlled by a permissions table and activity is saved in an audit table.

The browser is the most untrusted part, because a user can change any request by hand. The API is the part that decides. The database must receive only parameterized queries so that user text can never become a SQL command. The upload folder can be touched only by the server, and only after the permission check. The admin audit log needs both login and the admin role.

The one rule behind the whole design is that the backend decides and the frontend only shows. Hiding a button in React is only for convenience and it is never a security control.

4. What needs protection

Item	What is needed
User passwords	Never stored as plain text
JWT tokens	Must not be stolen or misused, because whoever has the token acts as that user
Document contents	Only authorized users can read them
Document details	Name, category and owner should not be readable or changed by others
Sharing permissions	Only the right person can give or remove access
Audit logs	Restricted and not easy to change or delete
Secrets and database credentials	Must stay out of GitHub
Uploaded files on disk	Must not be reachable without the permission check

5. Who may attack

- A person who is not logged in and calls protected endpoints.
- A logged-in user who changes document IDs or tries admin features.
- Someone who stole a password or token of a real user.
- A person who uploads very big or harmful files.
- An insider with admin power who misuses it or edits the logs to hide something.
- A person with access to the server or the database.

6. Threats

Threat 1: Access without login

What can happen. Someone sends requests to protected endpoints without a token and gets private data.

What I did. JWT middleware protects the protected routes. A request to the admin audit log without login is refused.

How it can be checked. There is an automated test for it, and you can also send a request without a token and see the 401.

What is still weak. I must check every route again to be sure the login check is on all of them. One forgotten route is enough for a leak.

Threat 2: One user opening another user's document

What can happen. A logged-in user changes the document ID in the request and reads someone else's file. This is the most serious risk for this kind of project.

What I did. The backend checks ownership and the permission table on the server before every protected document action. If neither exists, it answers 403.

How it can be checked. Automated tests check that cross-user read and download are denied. The audit log also shows many denied reads and downloads that were recorded while I tested.

What is still weak. Every document endpoint must use the same check, including edit, search, permission changes and delete. My tests cover only some of them.

Threat 3: Normal user using admin features

What can happen. A viewer tries to reach admin only functions and reads the audit log.

What I did. The audit endpoint needs login and the admin role. The role is read from the database and not only from the token.

How it can be checked. Tests send requests with no login and with a non-admin user, and both must be refused.

What is still weak. The way roles are assigned must also be protected, and I need to confirm that no route lets a user change their own role.

Threat 4: Downloading a file without permission

What can happen. A user guesses an ID or asks for a file path directly and gets a private file.

What I did. Downloads go through the permission check first. Files are in a private folder with random UUID names and are not served as public files.

How it can be checked. Tests for unauthorized download. Also check that no static route points to the upload folder.

What is still weak. File path handling needs continuing review. No route should let the user choose a path on the server.

Threat 5: Bad uploads

What can happen. A user uploads a huge file, or a harmful file renamed to look like a PDF or an image.

What I did. Multer allows only PDF, TXT, JPEG and PNG, up to 5 MB each. Saved names are random.

How it can be checked. Try a wrong type of file and a file bigger than 5 MB. Both should be rejected.

What is still weak. Checking type and extension is weak. I did not add file signature checks, quotas or malware scanning yet.

Threat 6: SQL injection

What can happen. An attacker types special text in a form or URL so that the database query changes.

What I did. The MySQL driver is used with parameterized queries.

How it can be checked. Read the query code and try quote characters in inputs. They should be treated as plain text.

What is still weak. All queries must be reviewed, especially the ones with dynamic sorting or filters like the audit log filters.

Threat 7: Stolen or weak passwords

What can happen. An attacker steals stored passwords or guesses an easy one.

What I did. Passwords are hashed with bcryptjs. The hashes in my database use cost factor 12.

How it can be checked. Look at the users table and confirm there are only hashes and no readable passwords.

What is still weak. Hashing does not help with weak passwords or phishing. There is no login rate limit and no password rule yet.

Threat 8: Stolen token

What can happen. An attacker gets a valid JWT and replays it.

What I did. Protected requests need the token and the role is checked from the database.

How it can be checked. Review where the token is created, stored and how long it lives.

What is still weak. Token expiry, safe storage in the browser, HTTPS, revoking tokens and changing the secret still need to be verified. A stolen token may work until it expires.

Threat 9: Changing the audit log

What can happen. A person with high access edits or deletes log rows to hide what they did.

What I did. Important actions and denied attempts are recorded. Only admins can open the audit view. Foreign keys use ON DELETE SET NULL so logs stay even if users or documents are deleted.

How it can be checked. Check the routes and confirm there is no way to edit or delete log rows from the application.

What is still weak. Database rows are not tamper proof. Anyone with direct database access can change them. Better options are append only storage, limited database rights, copies of logs elsewhere and hash chaining.

Threat 10: Leaking secrets

What can happen. Passwords, tokens, documents or error details leak through GitHub, API responses or logs.

What I did. Secrets are in environment variables. Git ignores .env files, uploads, logs, SQL exports and the database certificate. Helmet adds security headers.

How it can be checked. Read .gitignore and the staged files before every push.

What is still weak. Error messages, logs, browser storage and deployment settings still need review. A secret that was ever pushed to GitHub must be treated as leaked, even if deleted later.

Threat 11: Requests from other websites

What can happen. Another website makes requests to the API from a visitor's browser.

What I did. CORS allows only the frontend address.

How it can be checked. Call the API from a different origin and see that the browser blocks it.

What is still weak. CORS is not login or permission. If cookies are used for login in the future, CSRF protection is also needed.

Threat 12: Losing the stored files

What can happen. The hosting platform resets its disk, or someone deletes the upload folder.

What I did. Files and database details are stored separately, so at least the mismatch can be noticed.

How it can be checked. Check what the hosting platform does with the disk when it redeploys.

What is still weak. There is no backup, copy or recovery plan. This must be solved before any real use.

7. Example of attack attempts

Take the same example as before. User A shares salary.pdf with User B as reader.

What happens	Expected result
A downloads the file	Allowed, logged as success
B downloads the file	Allowed because of the read permission
C downloads the file	403, logged as denied
B tries to edit the file	403, because B has read only
Someone with no token calls the API	401
A removes B, then B tries again	403 on the very next request

The last row is the one I care about most. Some systems keep trusting a user for a while after the access is removed. TrustVault checks the permission again on every request, so the removal works at once.

8. What the audit table shows

My database export (24 October 2026) has 162 audit records that were created while I was building and testing. They give a real idea of how the controls behaved.

Action and result	Count
document_read, denied	37
document_download, denied	37
admin_document_download, denied	23
permission_grant, success	35
permission_revoke, success	18
document_upload, success	5
document_download, success	4

Action and result	Count
admin read, download and delete, success	1 each

The many denied reads and downloads show that the checks were used again and again and that refused attempts were recorded. I take this as supporting evidence only, not proof of security, because most of these rows probably came from my own testing.

9. Things I still need to check

While writing this I noticed some points that I should settle before saying the design is complete.

- Admin policy. I said admin should not get unlimited access to documents, but the log has separate admin document actions, some allowed and some denied. I need to write the exact rule and confirm that the code follows it.
- Who can grant permissions. In my test data at least one permission was granted by an admin account on a document owned by another user. I need to decide if this is a planned admin feature or if only owners should do it.
- Deleting documents. The permission table has foreign keys without cascade, so deleting a document that still has permissions must be handled carefully.
- Role changes. I must confirm that no endpoint lets a normal user change their own role.
- Error messages. They should help the user but not show internal paths or database details.

10. Risk summary

The ratings are my own judgement and are not a formal score.

Threat	Likelihood	Impact	Remaining risk
Access without login	Medium	High	Route coverage must be checked
Cross-user access	High	High	Every endpoint needs review
Admin feature misuse	Medium	High	Role assignment protection
Unauthorized download	High	High	File path handling review
Bad uploads	Medium	Medium	Harmful content may pass
SQL injection	Medium	High	Dynamic filters need care
Weak passwords	Medium	High	No rate limiting yet
Stolen token	Medium	High	Theft and revoke limits
Log tampering	Low	High	Rows are not immutable
Leaked secrets	Medium	High	Mistake while committing
Other website requests	Low	Medium	CSRF if cookies are used

Threat	Likelihood	Impact	Remaining risk
Lost files	Medium	Medium	No backup or recovery

11. Security testing

I used Jest and Supertest to test selected cases on the real Express app. The tests check that a user cannot read another user's document, cannot download it, that the audit log endpoint refuses requests with no login, and that it refuses a login without the admin role. Some filter validation cases are also included where they are in the current test suite.

I need to run these tests again on the final code before submission. Passing tests only prove the cases that were tested. They are not a certificate that the application is secure.

12. Not covered

- Attacks on the hosting platform, the operating system or the network.
- Large scale denial of service.
- Physical access, phishing and social engineering.
- Problems inside third party packages. I should run a dependency scan, but I do not count it as done.

13. What to improve, in order

1. Review every route for login and object level permission, and add tests for upload, edit, permission change and delete.
2. Add login rate limiting and password rules, and review token expiry and secret handling.
3. Decide and write the admin policy and the rule for who can grant permissions.
4. Add file signature checks, size quotas and malware scanning.
5. Move to persistent private storage with backup and recovery.
6. Protect the audit log with database rights, outside copies and hash chaining.
7. Use HTTPS and proper secret management in deployment, run a dependency scan and, if possible, get an independent review.

14. Conclusion

TrustVault shows how login, permission checks, private file storage and audit logging can work together in a small application. The main point I want to show is that the backend decides access on every request, and not the frontend.

The project handles several important threats, and I have tried to be clear about the ones it does not. More testing, a safer deployment, persistent storage and a protected audit log are

needed before anyone should trust it with real sensitive documents.